FINAL PROJECT REPORT

Apex Care Technologies: from a static brochure site to a full-stack ordering and management platform

Design, implementation and evaluation of a database-driven website, a quote-basket ordering system and a role-based administration panel for an industrial safety supplier, built with Next.js 16 and MongoDB.

Submitted by	**Manmay Chakraborty**
Registration no.	**24BAI1316**
Date	**28 October 2026**

84
API operations
29 public · 55 admin

28
pages and screens
10 public · 18 admin

18
MongoDB collections
plus GridFS storage

362
automated tests, all passing
65 in a real browser

Contents

1. Abstract

2. Introduction and motivation

3. Objectives

4. Analysis of the existing website

5. Requirements

6. Proposed system and architecture

7. Technology choices and justification

8. Database design

9. Features implemented

10. System walkthrough

11. Security and privacy

12. Comparison: existing vs. proposed

13. Testing and validation

14. Deployment

15. Limitations and future work

16. Conclusion

A. Appendix A: API summary

B. Appendix B: Running the project

1. Abstract

Apex Care Technologies supplies fire-safety, personal-protective, welding and gas-detection equipment from Cuddalore, Tamil Nadu. Its existing website is a set of hand-written static HTML pages: content is typed into the page files, an enquiry is posted to a single server script, and the business cannot edit content, take orders, follow up leads or measure visitors without a developer.

This project rebuilds the website as a full-stack application. One **Next.js 16** application serves the public website, a documented **REST API** and a sign-in protected **administration panel**; **MongoDB** is the only data store. Customers can browse a database-driven catalogue, collect products and quantities in a **quote basket** and send it as an order without online payment, receive a reference number, and track progress. Staff work in an 18-screen panel: an order desk, an enquiry inbox, content editing with a media library, testimonial moderation, site settings, analytics, user management, an audit log and backup and restore. Access is governed by three roles and fifteen permissions.

The system provides 84 API operations over 18 collections and is verified by **362 automated tests**, all passing, of which 65 drive a real Chrome browser. This report describes the problem, the requirements, the design, the features with screenshots, the security model, the testing, and how the system is deployed on MongoDB Atlas. It also states plainly what is not yet done: production deployment and email delivery are prepared but not switched on, and products are sold as categories, without individual prices or stock.

2. Introduction and motivation

For a small industrial supplier, the website is the main route by which new customers find the company and ask for a price. Buyers of safety equipment usually know what they need (helmets for a plant crew, a breathing apparatus, a gas detector), compare options and then ask for a quotation, often for several items at once. The site therefore has four jobs: present the catalogue accurately, make it easy to ask for a quote or place an order, let the business respond quickly, and keep a record of every lead.

2.1 Why a rebuild was needed

- **Every content change needs a developer.** Adding a product, correcting a phone number or publishing a training programme means editing HTML files and uploading them again.
- **No way to order.** A customer who needs several products must describe them in free text, and nothing is recorded as an order the business can work through.
- **Leads are fragile.** An enquiry is not given a reference number, cannot be followed up by the customer and has no status.
- **No visibility.** The business cannot see how many people visit, which products they view, what they search for or how many visitors become leads.
- **Weak discoverability.** Products have no pages of their own, so they cannot be found, linked or shared, and search engines receive little structured information.
- **Open to abuse.** Public forms attract automated spam; without defences the sales inbox fills with junk and real leads are lost.

2.2 Scope of this project

The look and page set of the original site were kept recognisable so the business is not asked to relearn its own website. The work went into what lies underneath and behind it: a real data layer, a secure API, the business

workflows around orders and enquiries, an administration panel that non-technical staff can use, and the testing needed to trust all of it.

2.3 How the project was carried out

The work proceeded in three stages, each finished and tested before the next began: first the data layer, API and database-driven public site; then the administration panel over the API; and finally quote-basket ordering, which added a new order collection, public checkout and tracking pages, and an order desk in the panel. Each stage extended the automated test suite, which is why the final system can be changed with confidence.

3. Objectives

1. Store **all** site content in a database and render every page from it, so that an edit appears on the live site immediately.
2. Give each product, survey and training programme its own addressable page with search-engine metadata.
3. Let customers **order several products in one go**, with quantities and delivery details, without online payment, and track the order afterwards.
4. Capture every enquiry reliably, number it, notify the team, acknowledge the customer and let the customer check progress.
5. Protect public forms against spam and abuse without inconveniencing real visitors.
6. Provide a **graphical, role-based administration panel** for orders, enquiries, content, settings and users, protected so that only signed-in staff can reach it.
7. Measure site usage without tracking individuals, respecting visitor privacy signals.
8. Make the system easy to run, back up and deploy, and demonstrably correct through automated testing.

4. Analysis of the existing website

The live site at apexcaretechnologies.com was inspected through its public pages, response headers and the scripts it serves. Only what is observable from outside is reported; the server-side behaviour of its enquiry script cannot be inspected and is not assumed.

Aspect	Observation on the existing site
Construction	Static HTML files (`index.html` , `products.html` , `survey.html` …) with Bootstrap and jQuery; content is written into the page files.
Product pages	All product categories link to one general `products.html` ; there is no page, address or search entry per product.
Ordering	None. There is no basket and no way to request several products together other than describing them in a message.
Enquiry submission	A browser script validates the form and posts it to `enquiry.php` . Phone validation accepts exactly ten digits (`/^\d{10}$/`), so `+91` or international numbers are rejected. There is no email field, so the customer cannot be answered or acknowledged by email.
Enquiry follow-up	The confirmation says only “We’ll be in touch”. No reference number, status or tracking page.
Spam control	No honeypot, token or rate limit is visible in the client code.

Aspect	Observation on the existing site
Search	None.
Search-engine files	`/sitemap.xml` and `/robots.txt` both return 404; no structured data on the home page.
Contact details	Email addresses are hidden by Cloudflare's obfuscation script and are not part of any editable settings.
Administration	No administration area or content-management system was found.
Analytics, newsletter, testimonials	None visible.

Table 1. Observed characteristics of the existing website.

> **Conclusion of the analysis.** The existing site works as an online brochure but has no ordering, no content management, no lead management, no visibility into usage and limited discoverability. These are limits of its architecture, not of its design, and they are what the rebuild addresses.

5. Requirements

5.1 Users of the system

Actor	Goals
Visitor / customer	Find products and services, order several products with quantities, ask questions, book surveys and training, track an order or enquiry, leave a testimonial, subscribe to updates.
Sales staff (role *sales*)	See new orders and enquiries immediately, contact the customer, record notes, move each order through quote, confirmation, dispatch and delivery.
Content editor (role *editor*)	Keep products, surveys, training, gallery, logos and leadership up to date; manage images and the catalogue PDF; see traffic.
Administrator (role *admin*)	Everything above, plus users and roles, site settings, audit log, backup and restore, and deleting records.

Table 2. Actors.

5.2 Functional requirements

ID	Requirement
FR1	Every page is rendered from database content; an administrator's edit is visible without redeploying.
FR2	Each product, survey and training programme has its own page with metadata.
FR3	A visitor can add products with a quantity and an optional size/model note to a basket, review it, and send it with contact and delivery details. No price or payment is involved.
FR4	Every order and enquiry receives a unique reference number (ORD-… / ENQ-…), and the customer can check its status with that reference and their mobile number.
FR5	The team is notified of each new order and enquiry; the customer receives an acknowledgement when they give an email address.
FR6	Staff can list, filter, search, assign, annotate, change the status of, bulk-update and export orders and enquiries.

ID	Requirement
FR7	Staff can create, edit, publish, hide, reorder and delete all content types and upload images and PDFs.
FR8	Customer testimonials are held for moderation and published only after approval.
FR9	The administration panel is reachable only by signed-in staff, and each screen and action is limited by role.
FR10	The site records usage without cookies or raw IP addresses and reports visitors, top pages, products, searches and conversions.
FR11	All content can be exported and restored.

Table 3. Functional requirements.

5.3 Non-functional requirements

- **Security:** hashed passwords, server-side sessions, brute-force protection, CSRF protection, strict input validation, safe file handling.
- **Privacy:** no raw IP storage, honour Do Not Track and Global Privacy Control, automatic expiry of personal and operational data.
- **Usability:** responsive layout from phone to desktop, dark mode, no CAPTCHA for genuine visitors, plain-language admin screens.
- **Reliability:** failed emails are recorded and retried; a database outage returns a clean error instead of crashing.
- **Maintainability:** one codebase, one database, typed end to end, and a hermetic automated test suite.

6. Proposed system and architecture

The proposed system is a single deployable application. Next.js renders the public pages and the administration screens on the server and exposes the REST API from the same codebase; everything talks to one MongoDB database. There is no separate backend service, no third-party content-management system and no second data store to keep in step.

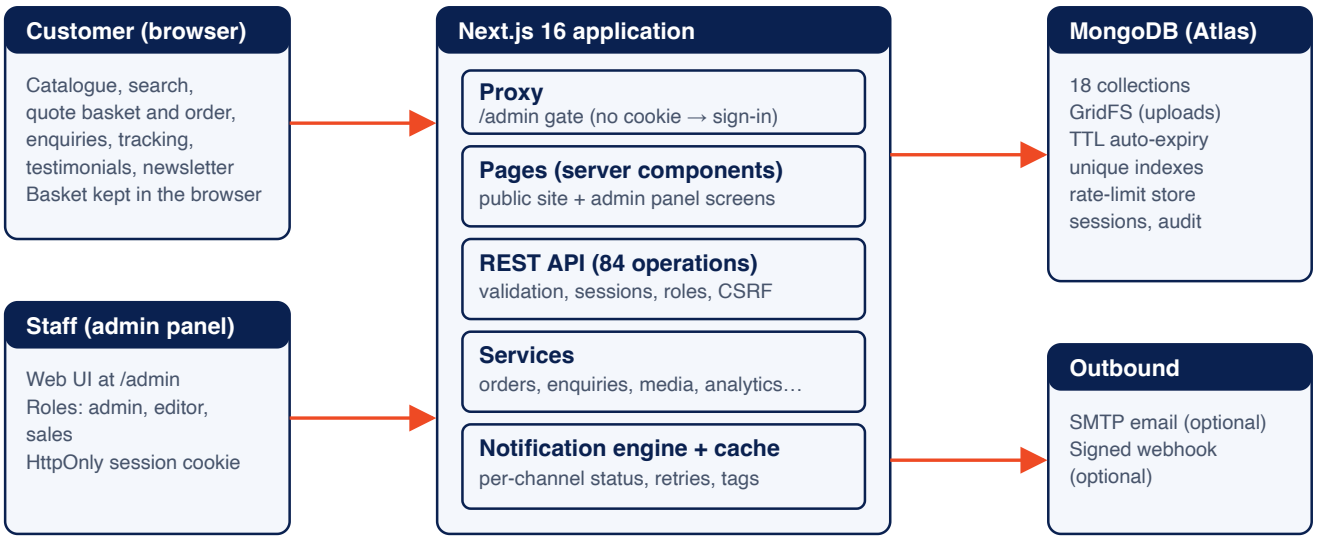


Figure 1. High-level architecture. Customers and staff use the same application; MongoDB is the single source of truth.

6.1 Layering

Layer	Responsibility
Public pages (`src/app/(site)`)	Home, About, Products, Product detail, Surveys, Survey detail, Training, Contact, Quote basket, Order tracking; 404 and error pages; `sitemap.xml` and `robots.txt` .
Admin panel (`src/app/admin`)	Sign-in page and 17 signed-in screens. Every screen checks the session and its permission on the server before rendering.
Proxy (`src/proxy.ts`)	Sends visitors without a session cookie to the sign-in page before any admin page renders, remembering where they were going.
API routes (`src/app/api`)	66 route files exposing 84 operations with a uniform response format and request ids.
Validation (`src/server/validation`)	Every request body and query string is checked against a strict schema; unknown fields are rejected.
Services (`src/server/services`)	Business logic: orders, enquiries, notifications, media, search, analytics, audit, backup, users, settings, dashboard.
Data (`src/server/db` , `data`)	Mongoose models and indexes; cached reads invalidated by tag when content changes.
Client state (`src/components/quote`)	The quote basket, kept in the browser's local storage so no account or cookie is needed; synchronised across tabs.

Table 4. Application layers and their responsibilities.

6.2 How an order is handled

The basket page fetches a short-lived signed form token. When the customer sends the order, the server checks the token and a hidden honeypot field, validates every field (quantities, Indian PIN code, optional GSTIN, mobile number), confirms that each product still exists and is published, merges repeated lines, scores the text for spam, applies per-address limits and collapses an accidental double submission. It then allocates a reference number atomically, stores the order and returns the reference. After the response, the sales email, the customer confirmation and an optional signed webhook are sent; each channel records its own status, attempt count and last error so a failure is visible and retried.

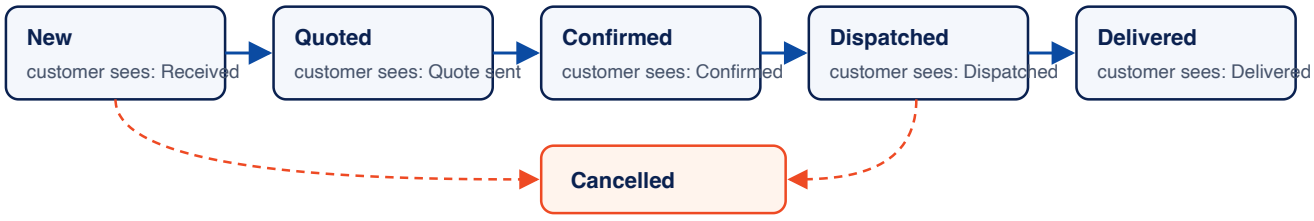


Figure 2. Order workflow. Staff move an order forward one step at a time; the customer's tracking page shows the matching label. An order can be cancelled from any open state.

7. Technology choices and justification

Technology	Role	Why it was chosen
Next.js 16 (App Router)	Website, admin panel and API in one application	Server rendering gives fast first loads and good indexing; route handlers remove the need for a separate API server; the Proxy file gates the admin area; one deployment.

Technology	Role	Why it was chosen
React 19	User interface	Component model suits cards, forms, tables and dialogs; server components keep database access off the client.
MongoDB 8 + Mongoose 9	Single data store	Catalogue items and orders with varied line items fit documents naturally; TTL indexes expire data automatically; GridFS stores uploads in the same database; MongoDB Atlas provides managed hosting.
Zod 4	Validation	One schema defines both the check and the TypeScript type, so the API contract cannot drift from the code.
TypeScript 5	Language	Type checking across API, services and UI catches mismatches at build time.
Tailwind CSS 4	Styling	Consistent design tokens, responsive layout, dark mode and print styles (used for packing slips).
Nodemailer	Email	Standard SMTP client; a local mail catcher is used in development so every email can be inspected without sending it.
Node crypto (scrypt, HMAC, SHA-256)	Security primitives	Built-in, audited primitives instead of extra dependencies.
Node test runner + Playwright core	Testing	API tests run against a real throw-away database; browser tests drive the installed Chrome.

Table 5. Technology stack.

7.1 Why one database for everything

An earlier plan used a hosted content-management service alongside a database. It was dropped: two stores mean two sets of credentials, two backup procedures and the risk of content, orders and enquiries disagreeing. Keeping everything in MongoDB gives one consistent system, one backup and one place to look when something goes wrong.

7.2 Why a quote basket instead of a shop with payments

Industrial safety equipment is priced by quantity, specification and site, which is why the original site shows no prices. A conventional shop would need a price, tax rate and stock level for every model and size, and online payment. A quote basket matches how the business actually sells: the customer states what and how many, the team replies with a quote, and the order moves through confirmation and delivery. It needs no new product data and can grow into priced ordering later.

7.3 Why build the panel instead of using a CMS

A ready-made CMS would give an editing screen quickly but would not provide the order desk, enquiry triage, tracking, spam handling or privacy-preserving analytics that the business needs, and it would bring its own maintenance and security burden. The panel is built directly on the tested API, so every action it offers is already validated, permission-checked and audited.

8. Database design

The database has 18 collections and a GridFS bucket for files. Slugs and reference numbers have unique indexes; operational collections carry TTL indexes so that old data removes itself.

Collection	Holds	Notable design points
products	Catalogue items (39)	Unique slug; belongs to a product group; featured flag, ordering, publish state, description, features, specification table, tags, gallery, SEO overrides
productgroups	Categories (5)	Cannot be deleted or re-keyed while products reference them
surveys · trainings	Surveys (9), training programmes (10)	Own pages, ordering, publish state
galleries · logos · leaders	Gallery, client and brand logos, leadership	Ordered, publishable
testimonials	Customer feedback	Public submissions stored unpublished; submitter email kept private
sitesettings	Singleton settings	Company details, phones, emails, address, map, social links, tax numbers, page text, catalogue file, notification recipients
orders	Quote orders	Reference ORD-YYYY-NNNNNN; line items (product slug, title at time of order, quantity, note); customer, company, GSTIN, delivery address; status history, assignee, internal notes, first-response time; per-channel notification state; spam rows auto-deleted after 30 days
enquiries	Leads	Reference ENQ-YYYY-NNNNNN, status history, priority, assignee, tags, internal notes, notification state; spam auto-deleted after 30 days
subscribers	Newsletter	Pending, confirmed or unsubscribed; unconfirmed rows expire after 14 days
adminusers · sessions	Staff and logins	Only a hash of each session token is stored; sessions expire automatically
analytics	Usage events	No raw IP addresses; daily-rotating visitor hash; expires after 400 days
audit	Change history	Who changed which fields (not the values); expires after 730 days
counters · ratelimits	Infrastructure	Atomic yearly counters for order and enquiry numbers; request windows stored in the database so limits hold across servers and restarts
media.files / media.chunks (GridFS)	Uploaded images and PDFs	De-duplicated by SHA-256; usage-tracked

Table 6. Collections.

8.1 Design decisions for orders

- Each line stores the product *title at the time of ordering* as well as its slug, so renaming or removing a product later never rewrites a customer's past order.
- Orders are separate from enquiries rather than a new enquiry type: they have line items, a delivery address and a different workflow, and keeping them apart keeps both models simple.
- A hash of the normalised basket, together with the mobile number, detects an accidental double submission within ten minutes.

8.2 Initial data

An idempotent seeding script loads the original site's content (39 products, 5 groups, 9 surveys, 10 training programmes, 8 gallery images, 6 testimonials, 19 logos, 3 leaders and the site settings), creates all indexes and creates the first administrator. Running it again restores anything deleted but never overwrites edits; the same script seeds the production database on MongoDB Atlas (section 14).

9. Features implemented

9.1 Database-driven public website

- Every page reads live data; home-page statistics (counts, years in business) are computed from the database.
- Reads are cached and invalidated by tag, so an edit made in the panel appears on the site, in the sitemap, in search and in form drop-downs without redeploying.
- Each product, survey and training item has its own page with title, description, canonical link and OpenGraph data; `Organization`, `Product` and `Service` JSON-LD; generated `sitemap.xml` and `robots.txt` (which keeps the admin area out of search engines).
- Responsive layout, dark mode, image lightbox, live search dialog, and a mobile action bar for call, WhatsApp, products, quote basket and directions.

9.2 Quote basket and ordering

- **Add to quote basket** on every product page, with a quantity stepper and an optional "size, model or colour" note. A badge in the header and the mobile bar shows how many products are in the basket.
- **Basket page** (`/quote`): change quantities and notes, remove lines, then send with name, mobile, optional email, company and GSTIN, delivery address (state list and 6-digit PIN code) and notes. No price or payment is involved.
- **Server-side checks:** every product must exist and be published (errors point at the exact basket line); repeated lines are merged; 1–100,000 units per line and up to 50 lines; GSTIN format; the same spam defences as enquiries.
- **Confirmation:** an on-screen reference such as `ORD-2026-000123` , an itemised email to the team (reply-to the customer) and, when an email was given, a confirmation to the customer with a tracking link.
- **Tracking** (`/quote/track` , also on the Contact page): Received → Quote sent → Confirmed → Dispatched → Delivered, or Cancelled. It needs the reference and the mobile number, and shows the items but never the address, notes or assignee.

9.3 Enquiry management

- Enquiry form on Home, Contact, product, survey and training pages, for questions, survey bookings and training requests. The subject is checked against real published items.
- Indian mobile numbers accepted with or without `+91` / `0` and normalised; international numbers accepted; email optional but verified.
- Reference numbers such as `ENQ-2026-000123` , allocated atomically; notifications by channel with status, attempts and last error; failed deliveries retried (up to five attempts) on demand or by a scheduled job.
- Customer tracking with reference and mobile number; a wrong reference and a wrong number give identical answers, so references cannot be probed.

9.4 Anti-spam without annoying customers

Defences are layered and silent. A signed form token proves the page was loaded and enforces a minimum fill time; a hidden honeypot catches simple bots; content scoring looks at links, spam vocabulary, disposable email domains and URLs in names; identical submissions within ten minutes collapse into one; and per-address burst and daily limits apply. A bot is told the submission succeeded, but spam is stored out of sight, never emailed and deleted after 30 days. Genuine visitors never see a CAPTCHA.

9.5 Testimonials, newsletter, search, media and analytics

- **Testimonials:** a public “Share your experience” form stores submissions unpublished; staff approve or reject them in the panel’s Reviews queue.
- **Newsletter:** double opt-in sign-up, one-click unsubscribe, identical responses for new and existing addresses, CSV export of confirmed subscribers and permanent erasure.
- **Search:** live, ranked search across products, surveys and training, with escaped queries and cancellation of stale requests.
- **Media library:** images up to 8 MB and PDFs up to 25 MB in GridFS; file type taken from the bytes, not the name; duplicates stored once; files still in use protected from deletion; the catalogue PDF is served through a tracked download link.
- **Analytics:** cookie-less, no raw IPs, daily-rotating visitor hash, bots dropped, Do Not Track and Global Privacy Control honoured. Reports visitors, page views, orders, enquiries and conversion, catalogue downloads, daily charts, top pages, products, surveys and searches, contact clicks, referrers and devices.

9.6 Administration panel

The panel at `/admin` is a complete graphical interface over the API, in the site’s own design, working from phone to desktop. Menus show only what the signed-in role may use, and badges count new orders, new enquiries and testimonials awaiting review.

Screen	What staff can do
Dashboard	New, open, ready-to-dispatch and delivered orders; orders per day; most-ordered products; latest orders; new and open enquiries, average first response, failed notifications, enquiries by status and type; quick actions.
Orders	Status tabs; search by name, company, mobile, reference, product, city or PIN; filter by product, assignee and dates; bulk status, assign and delete; CSV export with one row per order line.
Order page	Items and quantities, customer and GSTIN, delivery address with Copy and Open in Maps; one-tap Call, WhatsApp and Email; one-click next step; assignee; internal notes; activity timeline; notification status with resend; printable packing slip.
Enquiries	Inbox with the same filtering, bulk actions and export; enquiry page with the full message, contact buttons, status, priority, assignee, tags, notes, activity and notifications.
Content	All eight content types: search, filters, bulk publish/hide/delete, reorder, and editors for every field (images from the media library, galleries, feature lists, specification tables, icons, SEO).
Reviews	Testimonial moderation queue: approve publishes, reject deletes.
Media library	Drag-and-drop upload, preview, alt text, “used in” list, safe delete.
Site settings	Eleven sections (company, contact, address and map, social, registrations, home banner, about text, why-choose-us, values, catalogue PDF, notification recipients), each saved on its own.
Analytics · Subscribers	Traffic report for any date range; newsletter list with export and erasure.
Users · Audit log · Backup	Create users with a role, change role, deactivate, reset password; filterable audit log; content backup download, validated restore, cache refresh.
My account	Change password; list signed-in devices and sign out any or all others.

Table 7. Administration panel screens.

9.7 Authentication, protection of the admin area, and roles

- Email and password sign-in; passwords hashed with scrypt; policy of at least 12 characters mixing letters and digits, not common and not based on the email.
- **Two layers protect `/admin`.** The Proxy redirects any request without a session cookie to the sign-in page. Every admin page then validates the session against the database on the server and checks its own permission, so a forged or expired cookie never renders anything. The API checks the same permissions on every call.
- After sign-in the user returns to the page they asked for, but only to `/admin` addresses, so the sign-in page cannot be abused to redirect elsewhere.
- Opaque server-side sessions with a seven-day sliding expiry; staff can see and revoke their devices; changing a password signs out other devices.
- Three roles and fifteen permissions: *admin* (everything), *editor* (content, media, analytics) and *sales* (orders, enquiries, subscriber list, analytics). Only administrators can delete orders and enquiries. The last administrator and the acting user cannot be removed or demoted.

10. System walkthrough

The screenshots in this section were captured from the running application with a sample order and enquiry, which were deleted afterwards. Names, numbers and addresses in them are illustrative.

10.1 A customer places an order

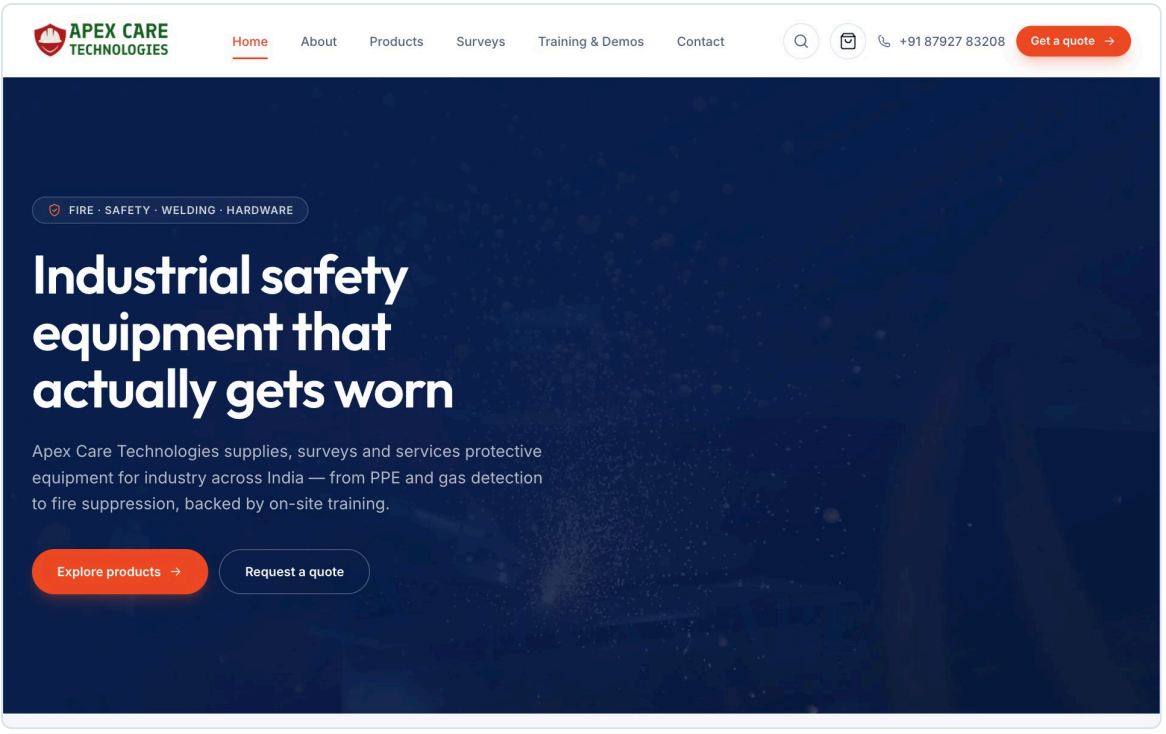


Figure 3. Home page banner with the main calls to action. The quote-basket icon sits in the header next to search.

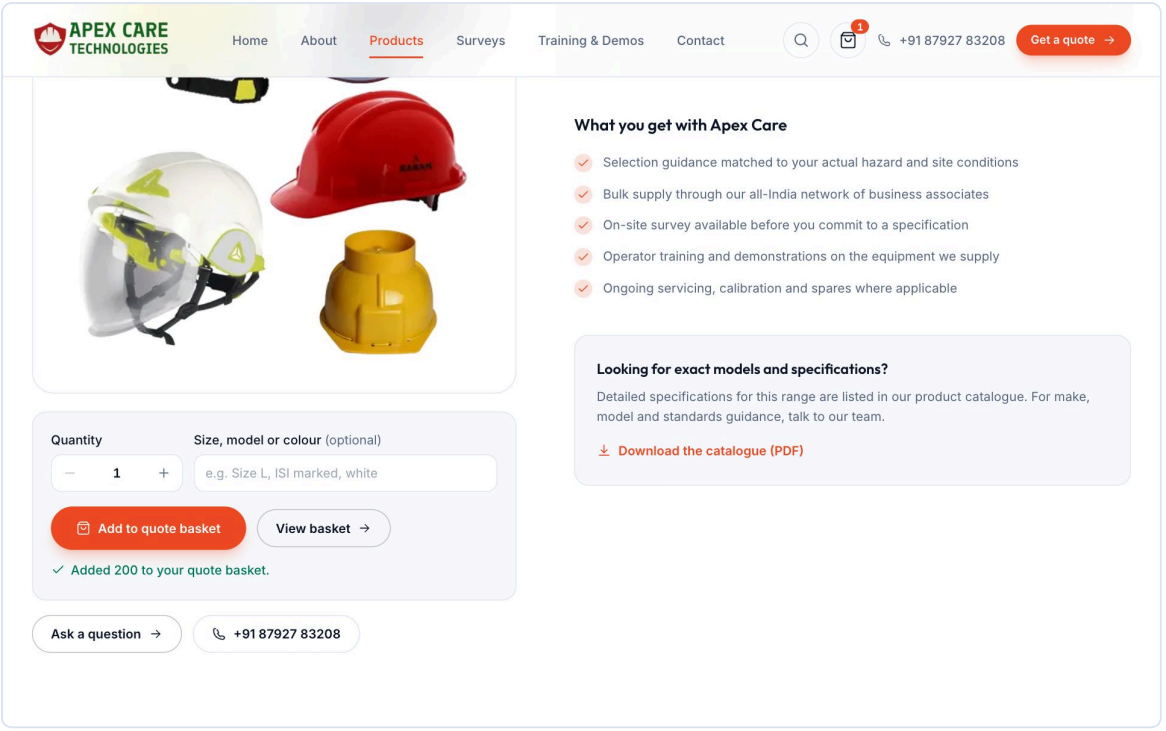


Figure 4. A product page. The customer chooses a quantity and an optional note and adds the product to the quote basket; the header badge counts the basket.

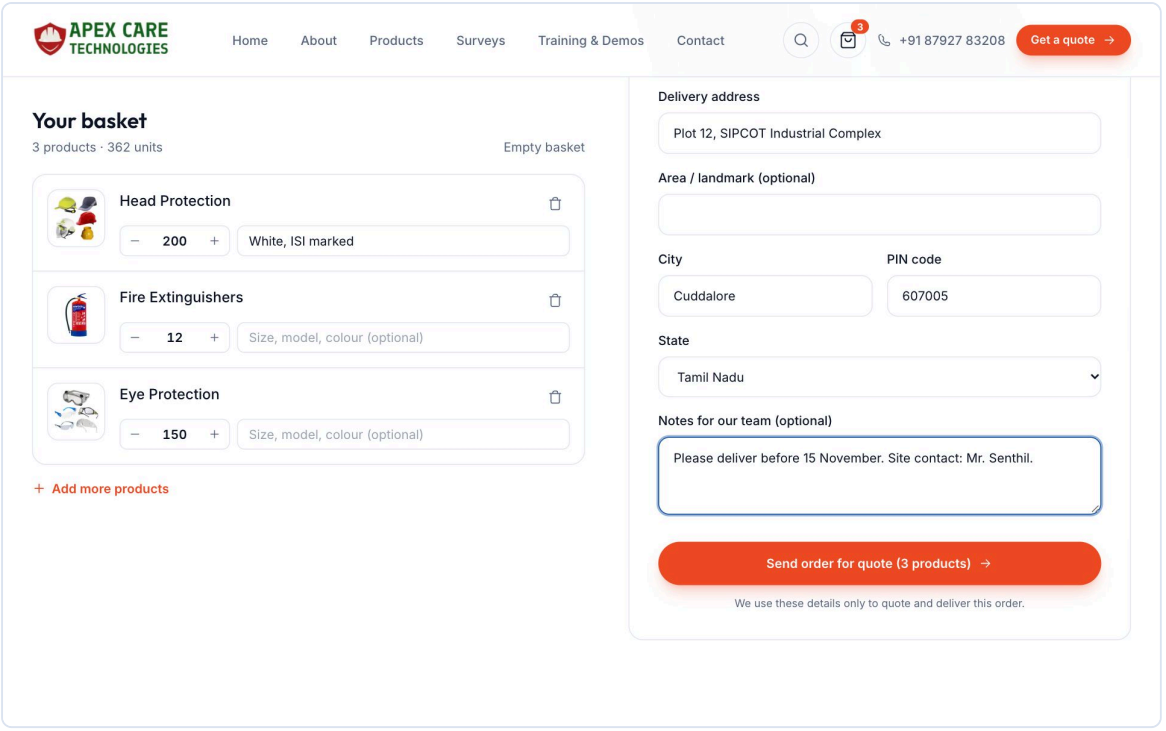


Figure 5. The quote basket. Quantities and notes can be changed beside the delivery and contact form. No payment is taken.

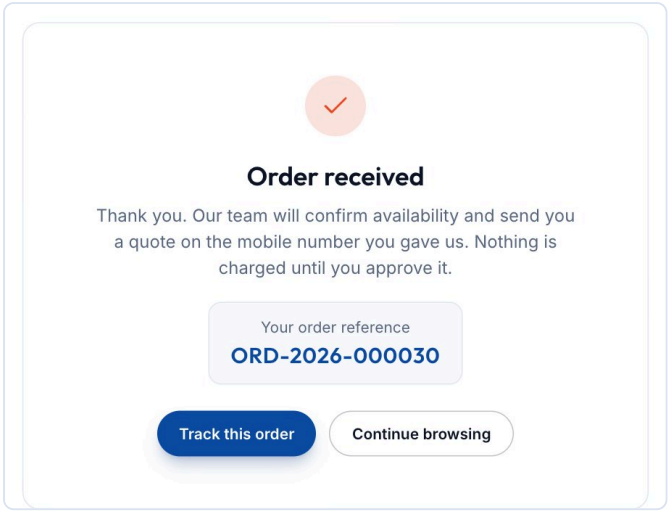


Figure 6. Confirmation with the order reference.

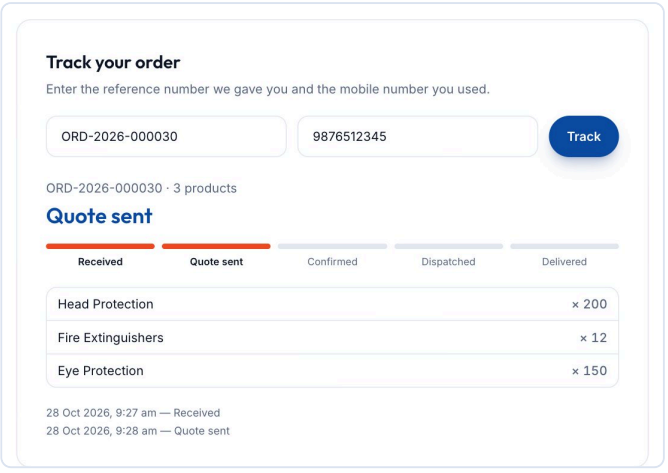


Figure 7. Order tracking after the team marked the quote as sent.

10.2 Staff handle the order

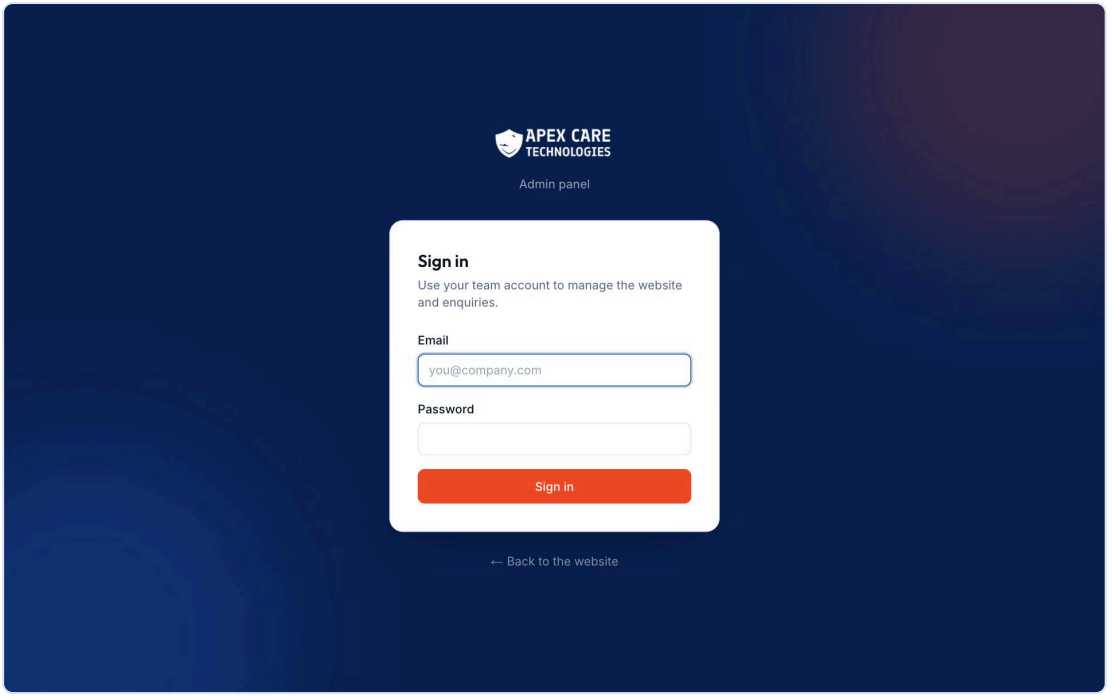


Figure 8. Sign-in page. Visitors who open any /admin address without a valid session are sent here first.

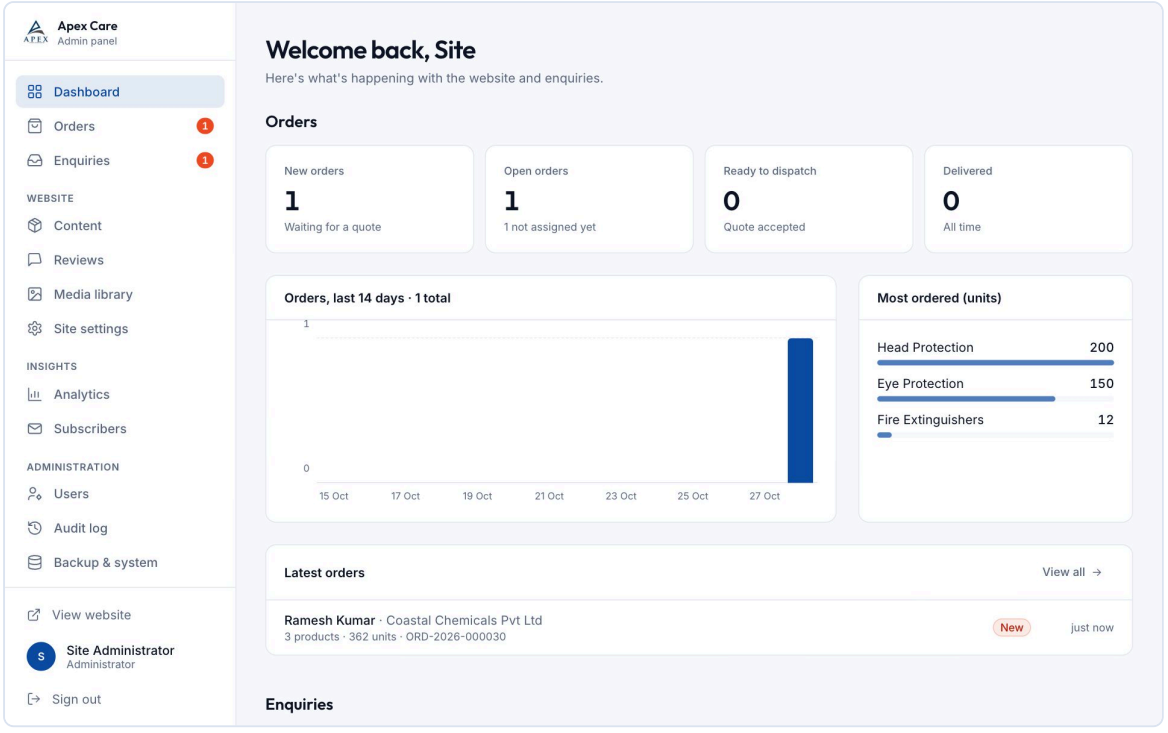


Figure 9. Dashboard: order tiles, orders per day, most-ordered products and the latest orders; menu badges count new orders and enquiries.

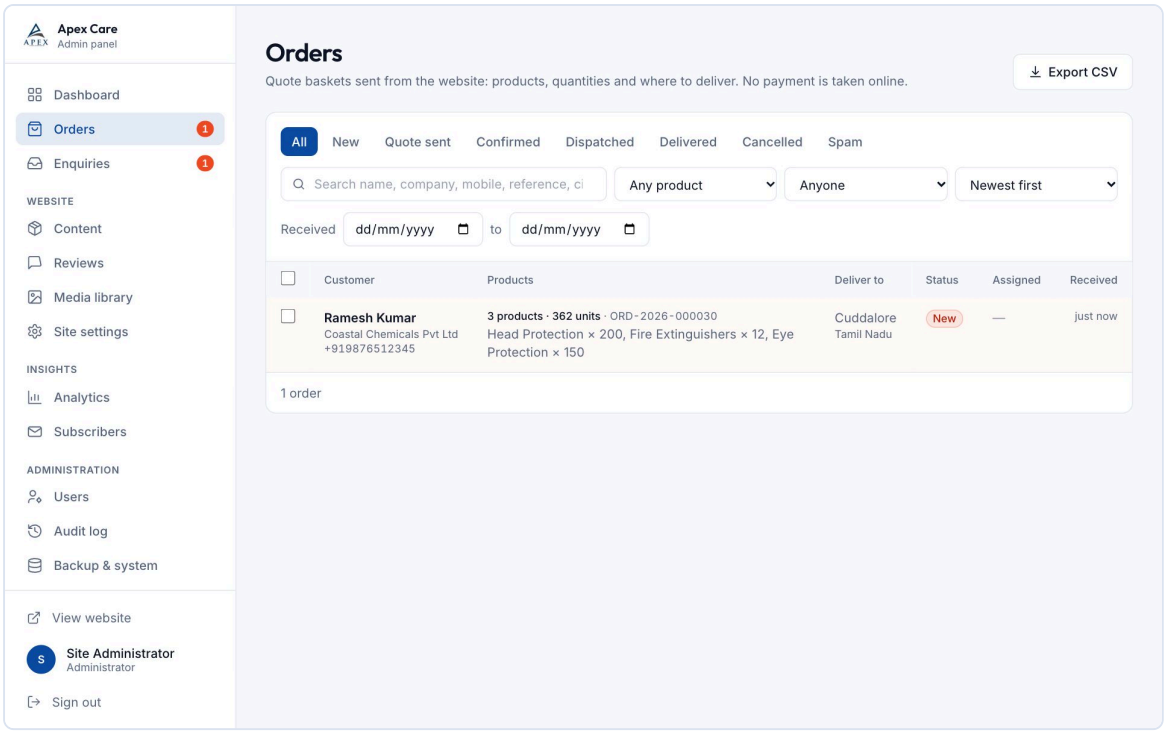


Figure 10. Orders desk with status tabs, search, filters, bulk actions and CSV export.

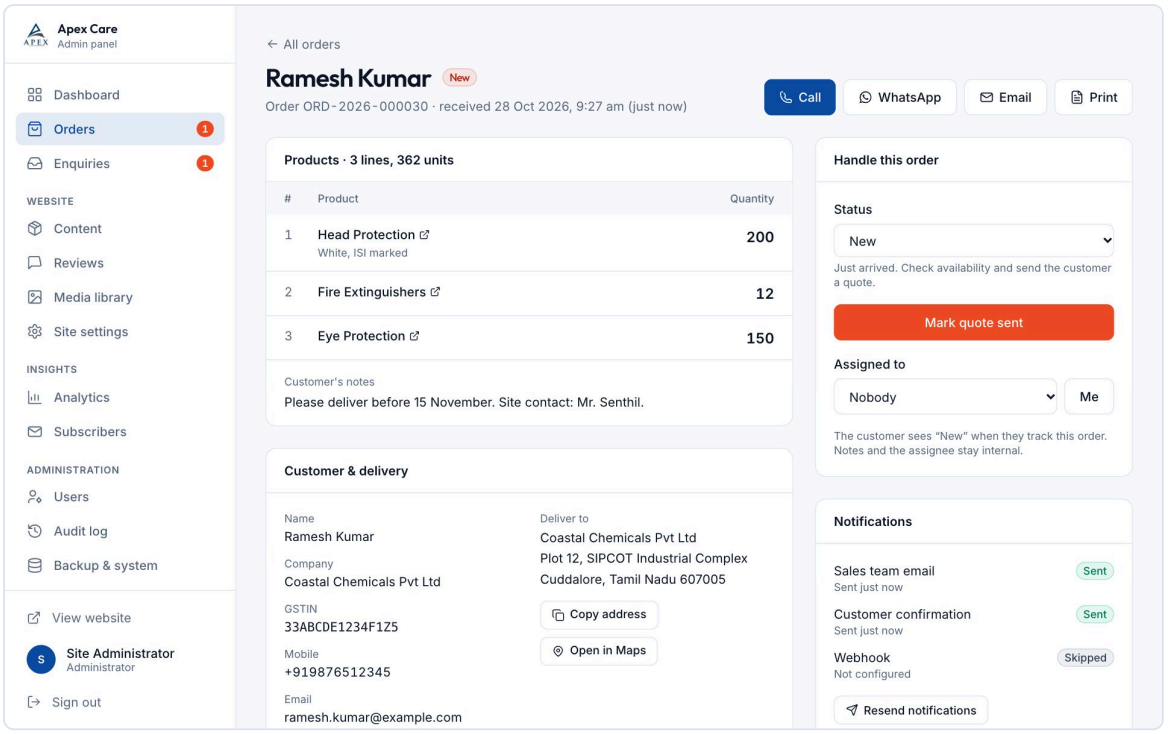


Figure 11. Order page: items, customer, GSTIN and delivery address, contact buttons, the one-click next step, assignee and notification status.

10.3 Enquiries, content and settings

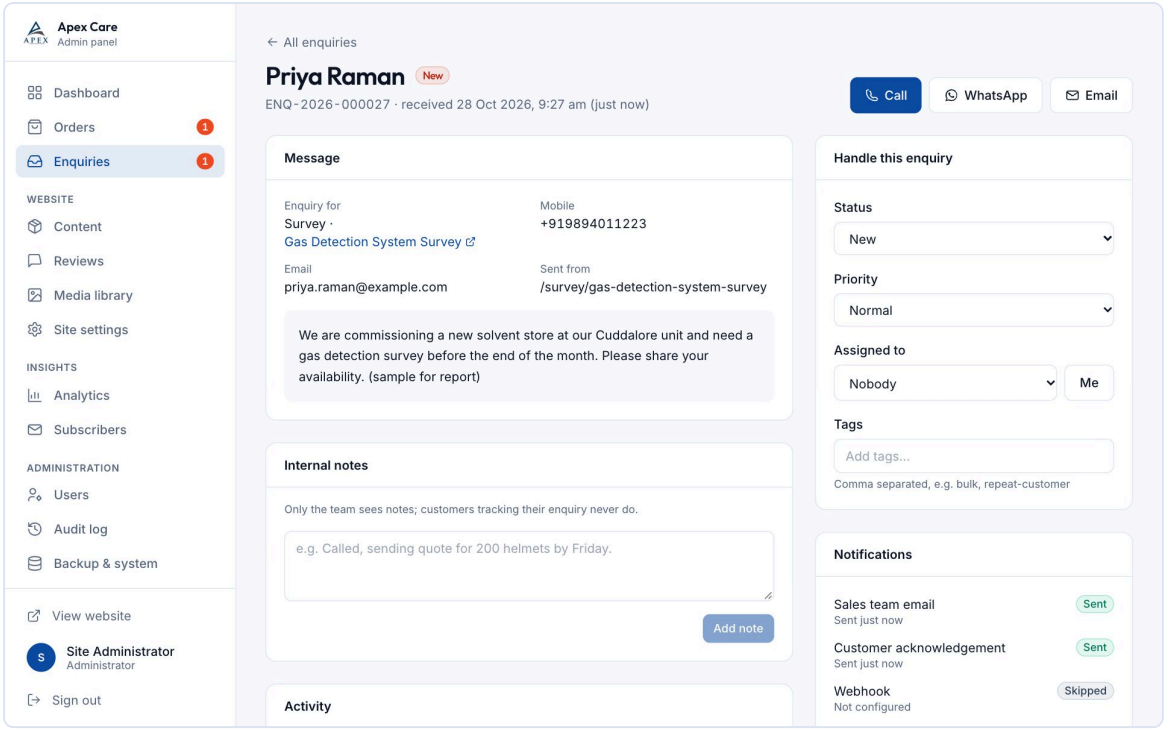


Figure 12. An enquiry (survey booking) with the customer's message, contact buttons and triage controls.

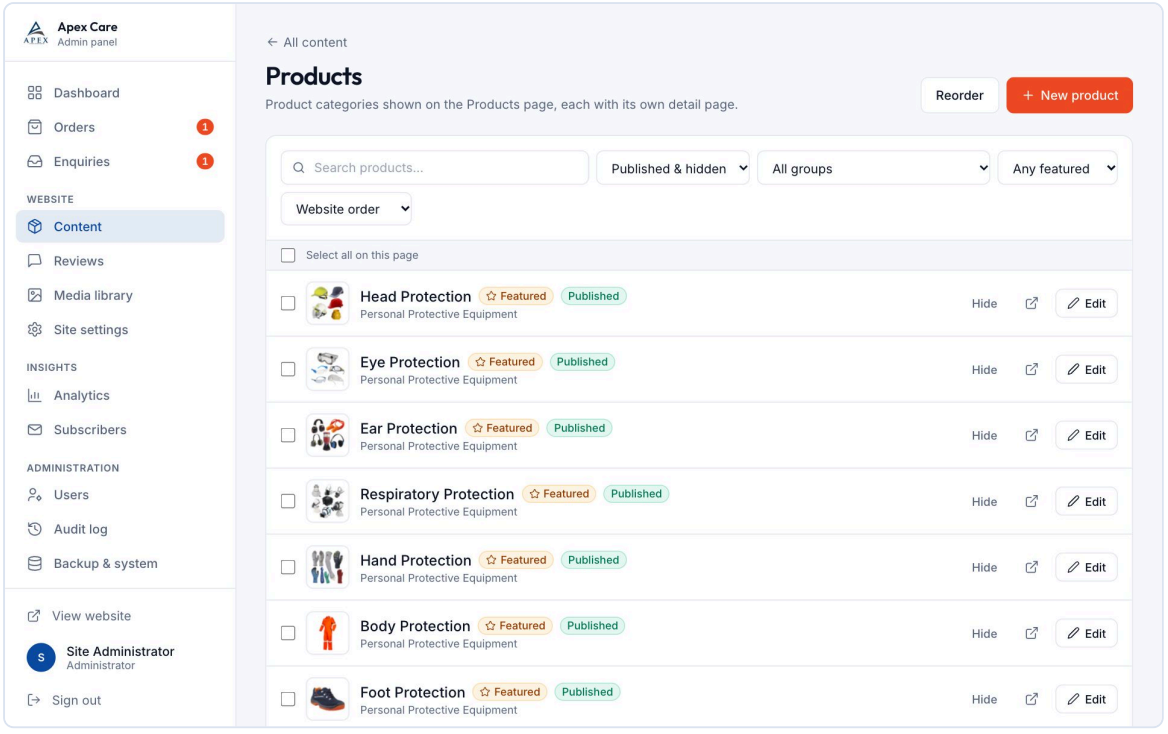


Figure 13. Content management: products with publish state, featured flag, bulk actions and reorder.

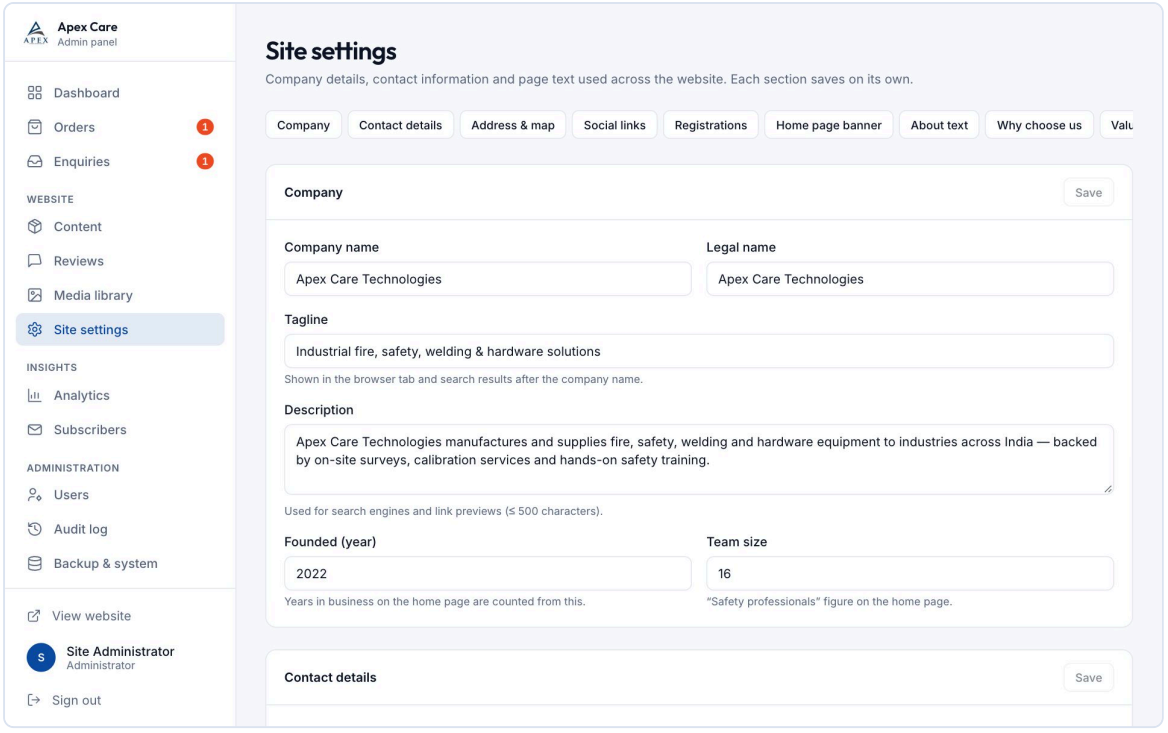


Figure 14. Site settings, saved section by section; changes appear on the website immediately.

10.4 On a phone

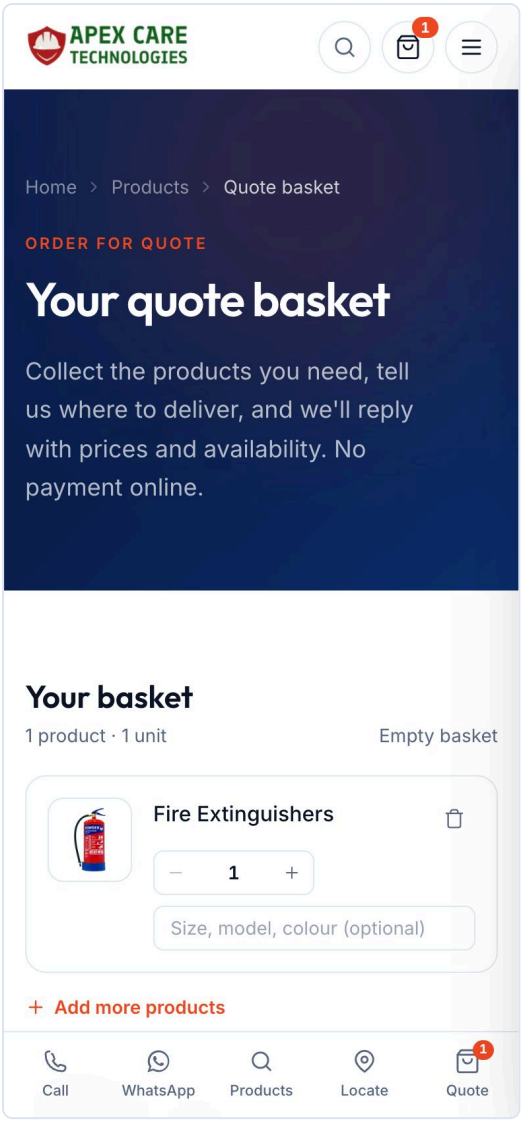


Figure 15. The quote basket on a phone, with the basket badge in the header and the bottom action bar.

Responsive by design. Every public page and admin screen was checked at phone width for horizontal overflow by the automated browser tests. The admin panel collapses its sidebar into a drawer on small screens.

11. Security and privacy

Threat	Measure in the system
Unauthorised access to the admin area	Proxy redirect for requests without a session; server-side session and permission check on every admin page; permission check on every admin API call; admin pages marked `noindex` and disallowed in `robots.txt`.
Password theft	scrypt hashing with per-user salt; hashes never returned; password change revokes other sessions.
Session theft	Random token of which only a SHA-256 is stored; HttpOnly, SameSite=Lax cookie (Secure over HTTPS); server-side revocation.
Brute force	Per-address limit, per-address-and-email limit and account lockout (10 failures, 15 minutes); unknown emails get the same error in comparable time, so accounts cannot be enumerated.
Open redirect	The post-sign-in destination is accepted only if it is an `/admin` path.

Threat	Measure in the system
Cross-site request forgery	Origin and Sec-Fetch-Site checks on cookie-authenticated writes.
Privilege escalation	Role-based permissions on every operation; deleting orders and enquiries is administrator-only; role changes and deactivation end sessions.
Injection	Strict schemas; typed queries so input cannot introduce database operators; escaped regular expressions; HTML-escaped email templates; header-injection guards; escaped JSON-LD; CSV formula neutralisation.
Tampered orders	The basket lives in the browser, so the server re-validates every line: product exists and is published, quantity in range, line count capped; the title stored is the server's, not the client's.
Information leakage through tracking	Order tracking needs reference and mobile; it never returns the address, notes or assignee; misses and mismatches are indistinguishable; throttled per address and per reference.
Malicious uploads	Content sniffing, size limits, executable/SVG/HTML refusal, `nosniff` and sandboxing CSP on served files.
Spam and flooding	Token, honeypot, scoring, duplicate collapsing and database-backed rate limits.
Excess data collection	No raw IPs; daily-rotating visitor hash; consent-based newsletter; erasure; TTL expiry of sessions, analytics, audit logs, spam and unconfirmed subscribers.
Outages	Database down returns a clean 503; failed emails and webhooks are recorded and retried; a health endpoint reports database and mail status.
Vulnerable dependencies	Production dependencies audited clean (`npm audit --omit=dev` : 0). During this work, new Nodemailer advisories were found and fixed by upgrading to 10.2.1.

Table 8. Threats and countermeasures.

12. Comparison: existing vs. proposed

The existing column describes what is observable on the live site (section 4). The proposed column describes what is implemented and tested in this project.

Capability	Existing site	Proposed system
Ordering several products	**Not possible**	**Quote basket with quantities, notes and delivery details**
Order tracking for customers	**No**	**Reference + mobile; five-step progress**
Order handling for staff	**No**	**Order desk: status workflow, notes, assignee, export, packing slip**
Content storage	In HTML files	**MongoDB; edits appear live**
Changing a product or phone number	Developer edits and uploads files	**A form in the admin panel; no redeploy**
Graphical administration	**None found**	**18-screen panel with roles**
Individual product pages	**No** (one shared page)	**39 products with own address and metadata**
Site search	**None**	**Live, ranked, cross-collection**
Phone number validation	Exactly 10 digits only	**Indian formats normalised to +91; international accepted**

Capability	Existing site	Proposed system
Reference number for an enquiry	**No**	**Yes, e.g. ENQ-2026-000123**
Staff lead management	Not visible	**Inbox with status, priority, assignee, notes, history, bulk, CSV**
Spam protection on forms	None visible	**Token, honeypot, scoring, rate limits (silent)**
Testimonials	**None**	**Public submission with moderation**
Analytics	**None visible**	**First-party, no cookies, no raw IPs, DNT honoured**
Search-engine optimisation	**No sitemap or robots.txt**	**Sitemap, robots, JSON-LD, canonical and OpenGraph per page**
User accounts and roles	**None found**	**3 roles, 15 permissions, audit log**
Backup and restore	Copy files	**Validated JSON export and all-or-nothing import**
Automated testing	None known	**362 tests including 65 real-browser tests**
Prices and online payment	**No**	**No, by design: orders are quoted (section 15)**

Table 9. Capability comparison.

Summary. The public look stays familiar. Behind it, customers can now order and track, the business can manage its own content and handle every order and lead from one panel, and the site can be found by search engines and measured without tracking individuals.

13. Testing and validation

Testing is automated and hermetic: `npm test` starts its own temporary MongoDB and mail catcher on free ports, seeds data, builds and serves the production application, runs every suite and removes everything afterwards, so no real data is touched. The final run passed **362 of 362** tests in under two minutes.

Suite	Tests	What it demonstrates
Unit	49	Slugs, phone parsing, CSV-injection guard, spam scoring, file sniffing, password hashing, tokens, permissions
Public API	38	Every public endpoint, filters, pagination, search, caching and security headers
Orders	25	Checkout with real emails, line merging, double-submit collapse, validation of lines, quantities, PIN and GSTIN, unknown products, honeypot spam, tracking (items shown, address hidden, cancellation), triage, notes, filters, bulk, CSV, dashboard and analytics, resend, role access
Enquiries	38	End-to-end pipeline with real emails, spam handling, rate limits, triage, notes, bulk actions, CSV
Authentication	33	Login, enumeration resistance, lockout, sessions, CSRF, role matrix, user management
Content	35	CRUD for all content types, instant public visibility, integrity rules, settings merge
Engagement	27	Testimonials, enquiry tracking, newsletter double opt-in, import and restore
Media	16	Upload, serving, ETag, hostile uploads, usage-aware delete, PDF catalogue
Operations	20	Analytics, audit log, export, scheduled retries, access control

Suite	Tests	What it demonstrates
Infrastructure	10	TTL and unique indexes, seed idempotency, hashing at rest
Webhook	6	Signature, failure then retry then success, attempt cap
Browser: public site	28	Every page free of console errors, broken images and overflow; search; forms; tracker; newsletter; testimonial; lightbox; mobile; dark mode; SEO tags; analytics; privacy signal
Browser: admin and ordering	37	Sign-in redirect and return; forged cookie rejected; no open redirect; noindex; sign-out; every admin screen loads cleanly; a customer fills the basket, orders, the order reaches the desk, moves to “Quote sent” and the tracker follows; enquiry triage; content edits going live; moderation; settings; media; role limits; mobile drawer

Table 10. Automated test suites (362 tests in total).

13.1 Defects found by testing and fixed

- **Navigation race in admin lists.** Browser tests showed that opening a row within a third of a second of typing in the search box could bounce the user back to the list, because the delayed search update rewrote the address while the router was navigating. The lists now flush the pending search when a row is opened and update the address through the History API, which cannot cancel a navigation.
- **Dependency advisory.** A pre-release audit found newly published advisories in Nodemailer’s address parser; the library was upgraded and the email-related suites (96 tests) re-run successfully.
- **Accessible labels.** Tests located form fields by their visible labels, which exposed a required-field asterisk leaking into the accessible name; it is now hidden from assistive technology.

13.2 Static checks

The TypeScript type check and the linter pass across the whole project with no errors or warnings, and the production build succeeds without needing a database.

13.3 What testing has not covered

Browser tests ran in headless Google Chrome only; other browsers, screen readers and real-world load have not been tested. Email was verified through a local mail catcher rather than a live mail provider.

14. Deployment

The production database is hosted on **MongoDB Atlas**. The application can run on any Node.js host (for example Vercel, Render, Railway or a virtual server) with build command `npm run build` and start command `npm start`, on Node 20.9 or later.

14.1 Configuration

Variable	Status	Purpose
`MONGODB_URI`	**Required**	Atlas connection string with `/apex` as the database name.
`SESSION_SECRET`	**Required**	At least 32 characters; signs form tokens and keys visitor hashes. Generated once and never changed after launch.
`PUBLIC_SITE_URL`	**Strongly recommended**	The https address; enables the Secure cookie flag and correct links in metadata and emails.

Variable	Status	Purpose
`SMTP_*`, `MAIL_FROM`, `SALES_EMAIL`	Optional (later)	Email delivery. Without them orders and enquiries are still stored, shown in the panel and trackable; notifications are recorded as skipped.
`CRON_SECRET`	Optional (with email)	Protects the scheduled endpoint that retries failed emails.
`TRUST_PROXY` and others	Defaults are correct	`TRUST_PROXY=true` suits hosts behind a proxy; retention periods and limits have sensible defaults.

Table 11. Production environment variables.

14.2 Steps

1. In Atlas, create a database user with read and write access and allow network access from the deployment host (and, for seeding, from the developer's machine).
2. Fill the production settings file (kept out of version control), including the first administrator's email and password.
3. Run the seed against Atlas: it creates every index and loads the catalogue, settings and first administrator. In production the application does not build indexes on start, so this step is required.
4. Set the environment variables on the host and deploy; sign in at `/admin`, change the password and add staff accounts.

> **Status at the time of writing.** The production settings file is prepared with generated secrets and the seed has been verified locally. Seeding Atlas and deploying the application are pending the Atlas connection string and the choice of hosting provider; email delivery is intentionally left off for now.

15. Limitations and future work

Limitation	Effect	Possible next step
Not yet deployed	The system runs locally; the Atlas database is not yet seeded.	Complete the steps in section 14.
Email not switched on	Orders and enquiries are stored and visible, but the team and customers receive no emails; newsletter sign-ups cannot be confirmed.	Configure an SMTP provider and schedule the retry job.
Products are categories	Customers describe size or model in a note; there are no per-model prices, tax rates or stock levels.	Add product variants with prices and stock, then optional online payment.
No customer accounts	Customers track orders with reference and mobile; there is no order history across orders.	Optional sign-in by one-time code sent to the mobile number.
Admin account recovery	No "forgot password" on the sign-in page; another administrator must reset it (or a command-line script).	Email-based password reset once email is on; optional two-factor sign-in.
Single-browser verification	Tested in Chrome only; no screen-reader or load testing.	Cross-browser, accessibility and load testing.

Limitation	Effect	Possible next step
Product copy is generic	Descriptions are the original category text; no certifications are claimed.	Add real descriptions, features and specifications through the panel.
Development-only advisories	Three moderate advisories remain in the local test mail catcher's dependencies; nothing ships to production.	Update when those packages release fixed versions.

Table 12. Known limitations.

16. Conclusion

The project turns a static brochure site into a maintainable, secure web application in which content, orders, leads and usage data live in one database, and in which the business runs its day-to-day work from a graphical administration panel. Each objective in section 3 is met: content is database-driven and live; every product has its own page; customers can order several products, receive a reference and track progress; enquiries are validated, numbered, trackable and protected from spam; the panel is protected at two layers and limited by role; analytics respect privacy; and the whole system can be run with one command and checked with 362 automated tests.

Compared with the original site, the clearest gains are in selling (an ordering path that matches how industrial equipment is quoted), in lead handling (reference numbers, acknowledgements, tracking, and order and enquiry desks), in discoverability (individual pages, sitemap, structured data and search) and in operability (editing without a developer, backup and restore, audit and monitoring). The remaining work is operational rather than architectural: deploying to the prepared Atlas database, switching on email, and, when the business is ready, adding priced product variants.

Appendix A. API summary

Group	Operations
Public content (12)	`GET /api/settings · stats · products · products/{slug} · product-groups · surveys · surveys/{slug} · training · gallery · testimonials · leaders · logos`
Public utility (4)	`GET /api/search · catalogue · media/{id} · health`
Public orders (2)	`POST /api/orders · orders/track`
Public enquiries and engagement (10)	`GET /api/enquiries/token` ; `POST /api/enquiries · enquiries/track · testimonials · newsletter/subscribe · analytics/events` ; `GET+POST /api/newsletter/confirm · unsubscribe`
Scheduled (1)	`POST /api/cron/notifications` (secret-protected; retries order and enquiry notifications)
Admin (55)	Authentication and sessions; users and assignees; **orders** (list, get, update, delete, notes, resend, bulk, CSV); enquiries (the same set); content for 8 types (list, create, get, update, delete, reorder, bulk); settings; media; subscribers; dashboard; analytics; audit; export, import and cache purge

Table 13. API overview (84 operations). The full table is in features.readme.md.

Appendix B. Running the project

Requirements: Node 20.9 or later (developed on Node 24). No Docker or system database is needed for development.

`npm install` then `npm run dev:all`, which starts a local MongoDB, a local mail catcher, seeds the database and runs the site:

- Website: `http://localhost:3000` · Admin panel: `http://localhost:3000/admin`
- Mail catcher (every email the site sends): `http://127.0.0.1:8025`
- Tests: `npm test` · type check: `npm run typecheck` · lint: `npm run lint`
- This report: `npx tsx scripts/build-project-report.ts`; its screenshots: `scripts/capture-report-figures.ts`

B.1 Project size

Measure	Value
Application source (TypeScript/TSX in `src/`)	about 19,200 lines
Automated tests (`tests/`)	about 3,700 lines, 362 tests
Public pages · admin screens	10 · 18
API route files · operations	66 · 84
Collections · roles · permissions	18 · 3 · 15

Table 14. Project size.

B.2 Project documents

`README.md` (how to run and deploy), `features.readme.md` (complete feature inventory and API reference), `.env.example` (every setting) and the source in `src/`, `scripts/` and `tests/`.

Report prepared by Manmay Chakraborty (24BAI1316), 28 October 2026. Observations of the existing website were made from its public pages and scripts and may change as that site is updated.